Enhancing Programming Skills in Audio Signal Analysis and AI: A Problem-Based Learning Approach

Tessa T. Taefi[1]

[1] *Smart Reality Lab, University of Applied Sciences Hamburg, 22081 Hamburg, Germany, Email: tessa.taefi@haw-hamburg.de*

Background

Challenges in teaching Artificial Intelligence (AI) programming include ensuring the school's readiness, enhancing teachers' AI competencies, and fostering students' ethical understanding and AI literacy [1]. Within the interdisciplinary fields of media engineering and media informatics these challenges extend the mere transfer of technical knowledge, as students enrolled in media programs typically come from diverse academic and practical backgrounds. This directly influences the design and delivery of AI-focused teaching modules.

Media technology students, for example, often possess a solid foundation in programming (e.g. Python) systems thinking and digital signal processing. They understand how audio and visual signals influence human perception and are adapt at aligning technical work with real-world applications.

Media informatics students, in contrast, tend to bring more advanced programming capabilities. They are proficient in object-oriented languages, skilled in debugging, and comfortable navigating technical documentation. Their understanding of Human-Computer Interaction enables them to design systems for usability and effective user experience.

To address this heterogeneity, our pedagogical approach is structured around four core competence areas, loosely inspired by the human-AI interaction framework proposed by [2]. These competency areas shape the design of learning activities and form the foundation for module-level learning objectives:

1 – Connecting to the Big Picture: Students draw from their domain-specific knowledge—such as acoustics, media data structuring, or visual systems—and apply theory-driven reasoning to contextualize problems within larger media systems. This fosters a user- and application-oriented mindset.

2 – Problem Solving and Data Understanding: Students tackle real-world tasks by curating datasets, making architectural design decisions, and developing evaluation strategies. This encourages inductive reasoning and critical data awareness.

3 – Programming and Model Implementation: Through hands-on work with tools like PyTorch, students gain technical fluency in feature engineering, optimization, and model design. This develops their bottom-up reasoning and experimentation skills.

4 – Fostering Future Skills: Problem-based learning scenarios promote creativity, collaboration, communication, and critical thinking [3]. These transversal skills are essential for navigating complex, technology-driven work environments [4].

By integrating these competency areas—spanning domain reasoning, data handling, technical depth, and future-facing soft skills—the modules aim to create a holistic and adaptive learning environment that prepares students for applied, reflective practice in AI-driven media development.

Methods

A key challenge in designing teaching modules for AI programming in media contexts is the constraint of time and credit points. All aforementioned competency areas must be addressed within a single elective module worth only 5 to 6 credit points, corresponding to 150 to 180 hours of student learning time—of which only about one third is spent in direct contact with the instructor. Consequently, there is insufficient time to explicitly teach all relevant technical content. Instead, the didactic approach relies on peer learning, collaborative knowledge exchange, and the cultivation of individual curiosity.

Students are expected to explore relevant subtopics more deeply during the module's self-directed learning phase, which accounts for approximately two-thirds of the total learning time. This process is guided by problem-based learning strategies that encourage intrinsic motivation, critical thinking, and active knowledge construction.

This study compares two distinct elective modules implemented in the Bachelor's curriculum at HAW Hamburg:

Module A "Smart Media Technology": Focused on programming and training neural networks to control MIDI parameters in a digital audio workstation using gestures or facial expressions. The module introduces students to the intersection of audio signal analysis, machine learning, and musical acoustics. The teaching strategy emphasizes hands-on learning through the implementation of neural networks in practical music technology applications. Students directly apply deep learning frameworks, such as PyTorch, to real-world scenarios involving multimodal input and real-time control systems.

Module B "Creative Audio Programming with AI": This module offers a broader conceptual scope, emphasizing the use of AI in audio applications ranging from beat tracking to generative sound synthesis. It integrates theoretical impulses from multiple domains—including audio signal processing, machine learning, and usability studies —and encourages students to independently explore the underlying methods (Figure 1).

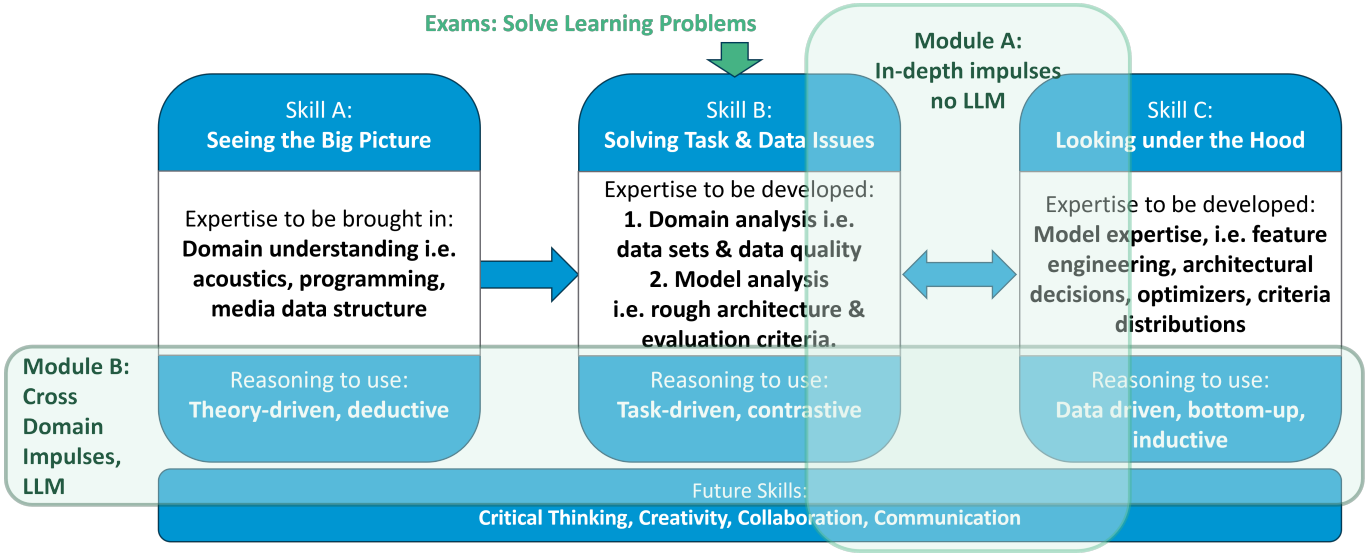

Figure 1: AI skills and competency areas, modules, and learning problems, loosely based on [2].

Participants and Structure

Both modules were conducted as in-person electives at HAW Hamburg and were aimed at Bachelor students in their final year. Each cohort included a mix of students from media technology and media informatics, bringing together diverse backgrounds in systems thinking, programming, and media application design. The modules focused on AI programming in media contexts and culminated in a portfolio-based examination. As part of this assessment, students formed groups of two to four persons and were required to solve two smaller and one larger problem-based learning tasks that formed the core of their individual performance evaluation. Despite their common framework, the modules differ in instructional design:

In Module A (Spring 2023), the course began with three basic "impulse problems" solved together in a code-along format led by the lecturer. These were followed by three small, peer-reviewed learning problems. Large Language Models (LLMs) such as ChatGPT based on GTP 3.5 were publicly available. Their use was not explicitly forbidden, but also not encouraged by the lecturer and an LLM-based coding assistant was not directly integrated in the coding environment.

In Module B (2024/25), the instructional design shifted. Instead of guided solutions, students received three cross-domain problem descriptions rooted in theory, without explicit coding templates. They were encouraged to use online resources, documentation, and tutorials—alongside peer feedback and LLM support by Gemini was embedded the coding platform Google Colab. This self-directed, exploratory learning environment emphasized autonomy, knowledge transfer, and responsible use of AI tools.

The study evaluates anonymous student feedback collected at the end of each module. The goal is to better understand how students perceive their learning progress, skill development, and the effectiveness of different teaching strategies in problem-based AI programming education.

Results & Discussion

Both modules successfully applied a problem-based learning approach to enhance programming and AI skills in audio-related applications. In both modules, the student teams chose their own final learning problem. However, differences in design, support structures, and available tools led to distinct learning experiences, summarized in Table 1.

Module A Projects: Gesture-Controlled Audio Interaction

All three projects in Module A explored gesture recognition as an interface to control sound in DAWs such as Cubase and were presented to the public at presented at the Universities campus exhibition:

- A live Solfège-based gesture control system that maps hand signs to MIDI instructions, enabling melody and harmony control in Cubase.
- A gesture-based audio control using CNNs trained on Mediapipe landmarks to trigger and modulate chord structures in a DAW.
- A gesture-based interactive storytelling game for children, integrating gesture recognition with audio responses via Cubase.

These projects show strong interdisciplinary engagement and creative outcomes. However, the transition from guided code-alongs to independent coding tasks created a notable gap in skill transfer for some students.

Module B Projects: Symbolic Music Generation with Transformers

Module B featured five individual projects focusing on symbolic music generation, exploring various transformer-based models. Students incorporated feedback trough designing and carrying out typical evaluation of creative systems surveys, such as the system usability

Table 1: Comparison of Module A and Module B

Aspect	Module A	Module B
Semester	Spring 2023	Winter 2024/25
Initial Impulses	Three code-along exercises	Three theory-driven, open problems
Project Guidance	Code-alongs + peer and lecturer review	Independent research + LLM-assisted development + peer and lecturer review
LLM Support	Neither encouraged nor forbidden	Required for concept explanations and coding assistance
Project Diversity	Three gesture-controlled DAW projects	Five symbolic music generation projects
Feedback Opportunity	Demand for more trial-and-error with feedback	Lacked feedback time and depth for complex, state-of-the-art topics
Main Challenge	Steep learning curve post code-alongs	Lack of expertise when facing unexpected model behavior

scale, creativity support index or technology acceptance model:

- A transformer-driven Drum-Groove Generator using the T5 model and human-centered design to extend simple MIDI drum patterns with genre-specific complexity.
- A symbolic music generator for 8-bit-style game music using GPT-2, themed by virtual environments (e.g., "Woods", "Boss-Fight"), with beginner-friendly UI.
- A Lo-Fi Music Generator creating relaxed music via MIDI tokens and transformers, aimed at low-threshold use for non-technical users.
- A Gradio-based classical music generator built with GPT-2, targeting users with basic technical knowledge.
- A Chord Progression Generator based on user-defined styles and complexity, using the Niko Chord Dataset and user-centered design principles.

These projects demonstrated a high level of creativity and ambition. However, the open-ended nature led students to select cutting-edge challenges. As a result, when model-specific issues arose, many lacked the expertise to debug effectively. Presentation, lecturer and peer review formats provided support, but the diversity and complexity of five parallel projects exceeded the review capacity of the instructor within the course's limited time-frame.

Implications for Future Course Design

This comparison underscores the trade-offs between structured guidance and exploratory autonomy. While code-alongs provide a safe entry point, they risk limiting depth. On the other hand, exploratory, LLM-supported environments empower creativity but require scaffolding mechanisms, especially for troubleshooting advanced model behavior.

Time constraints within a 5–6 ECTS module necessitate thoughtful didactic compromises. Peer collaboration, well-structured project framing, and integration of tool-support learning environments (such as LLMs) appear essential. These conclusions align with the findings of [5], who underscore the importance of structured guidance when incorporating LLMs into educational settings. Their work highlights that well-designed project frameworks and peer-supported learning environments can help students benefit from LLMs while mitigating associated challenges. Building on this, the present study further suggests that instructor support should be scalable. This could be achieved through the use of staged checkpoints, expert consultations, or embedded mentoring formats, particularly to support interdisciplinary teams working on complex, self-defined problems. Ultimately, the potential time savings gained by delegating certain explanatory tasks to LLMs should not be used to increase the complexity of assignments. Instead, this time should be reinvested in more frequent and targeted mentoring opportunities and feedforward reviews.

Conclusion

This study highlights the complexity of designing effective AI programming modules within interdisciplinary media education. Given the limited scope of a 5–6 ECTS elective, it is unrealistic to expect comprehensive coverage of all technical, methodological, and social competencies required in this rapidly evolving field. Instead, teaching strategies must rely on fostering curiosity, collaborative learning, and engagement through problem-based exploration.

Both modules contributed meaningfully to the four core competency areas defined at the outset. They encouraged students to connect programming with broader media contexts, supporting theory-driven and deductive reasoning. By tackling real-world problems, students engaged in task-driven exploration, learned to curate data, and developed inductive reasoning skills. Moreover, they gained insights into the inner workings of neural network frameworks, such as PyTorch and transformer architectures, developing hands-on technical proficiency through implementation and experimentation. Finally, both modules promoted the development of future-oriented competencies, including creativity, critical thinking, communication, and collaboration, through project-based assessments and peer interaction. By en-

gaging students in such learning problems, educators prepare them for a rapidly evolving industry where AI plays a crucial role in shaping media experiences.

The comparison between the two modules reveals inherent trade-offs. Structured formats such as code-alongs provide accessible entry points but can constrain deeper exploration. In contrast, open-ended formats supported by Large Language Models empower students to work independently but require stronger scaffolding, particularly when projects involve complex or state-of-the-art systems. The introduction of LLMs in Module B allowed students to pursue ambitious projects more autonomously, yet also exposed limitations in their foundational understanding when technical challenges arose.

Future course iterations should aim to strike a balance between guidance and autonomy. Integrating structured feedback loops, peer teaching, and adaptive support tools will be essential to equip diverse student cohorts with the skills, confidence, and mindset needed to succeed at the intersection of AI and audio signal analysis.

References

[1] Ng, D.T.K., Chan, E.K.C. and Lo, C.K.: Opportunities, challenges and school strategies for integrating generative AI in education. Computers and Education: Artificial Intelligence 8, 2666-920X (2025). https://doi.org/10.1016/j.caeai.2025.100373

[2] El-Assady, Mennatallah. *Levels of Explainability for Human-AI Interaction in Visual Text Analytics*. Dissertation, Universität Konstanz, 2023.

[3] Ghani, A.S.A., Rahim, A.F.A., Yusoff, M.S.B. et al. Effective Learning Behavior in Problem-Based Learning: a Scoping Review. Medical Science Educator 31, 1199–1211 (2021). https://doi.org/10.1007/s40670-021-01292-0

[4] World Economic Forum. *Future of Jobs Report 2023*. Cologny/Geneva, ISBN-13: 978-2-940631-96-4.

[5] Jošt, G., Taneski, V., and Karakatic, S.: The Impact of Large Language Models on Programming Education and Student Learning Outcomes. Applied Sciences, 14(10), 4115 (2024). https://doi.org/10.3390/app14104115

Acknowledgments

The author used ChatGPT (OpenAI, Mar 2025 version) to support the refinement of phrasing and terminology. All content and interpretations remain the author's own.